

Aufgabe 3 (Schreibtischlauf)

(10 Punkte)

Welche Ausgaben sind bei folgendem Java-Programm möglich?

```
public class ThreadLocalTest {
    static volatile int a = 10;

    static class MyThread1 implements Runnable {
        public void run() {
            System.out.println("a=_ " + a);
            a = 30;
        }
    }
    static class MyThread2 implements Runnable {
        public void run() {
            System.out.println("a=_ " + a);
            a = 50;
        }
    }
    public static void main(String[] args) {
        new Thread(new MyThread1()).start();
        new Thread(new MyThread2()).start();
        new Thread(new MyThread1()).start();
    }
}
```

Mögliche Ausgaben (jeweils in ein Feld eintragen!):

Aufgabe 4 (Schreibtischlauf)

(10 Punkte)

Gegeben ist folgendes paralleles Java-Programm:

```
public class Output {
    static int a = 1;
    static int b = 2;
    public static void main(String[] args) throws Exception {
        Thread t1 = new Thread() {
            public void run() {
                a = 4; b = 5;
                String text = a + " < " + b;
                System.out.println(text);
            }
        };
        Thread t2 = new Thread() {
            public void run() {
                String text = b + " < " + a;
                System.out.println(text);
            }
        };
        t1.start(); t2.start(); t1.join(); t2.join();
    }
}
```

- a) Welche verschiedenen Ziffern sind in der Ausgabe als erste Ziffer möglich?
- b) Welche verschiedenen Ziffern sind in der Ausgabe als letzte Ziffer möglich?
- c) Welche verschiedenen, unwahren Ungleichungen könnte das Programm ausgeben?
- d) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden?
- e) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden, wenn die Variablen **a** und **b** als **volatile** deklariert werden?
- f) Mit wie vielen verschiedenen Ausgaben kann sich das Programm beenden, wenn die beiden **run**-Methoden als **synchronized** deklariert werden?

Aufgabe 4 (Schreibtischlauf)

(10 Punkte)

Gegeben ist folgendes paralleles Java-Programm:

```
public class Kaffee {
    private static int counter = 1;
    synchronized static String gib() {
        return "die_" + (counter++) + "_Tasse_Kaffee";
    }
    public static void main(String[] args) throws Exception {
        Thread anna = new Thread() {
            public void run() {
                System.out.println("Anna_verschuettet_" + gib() + "!!");
                System.out.println("Anna_trinkt_" + gib() + "!!");
            }
        };
        Thread berta = new Thread() {
            public void run() {
                System.out.println("Berta_trinkt_" + gib() + "!!");
                System.out.println("Berta_vergisst_" + gib() + "!!");
            }
        };
        anna.start(); berta.start(); anna.join(); berta.join();
    }
}
```

Hinweis: Nehmen Sie an, dass nur ein Thread auf einmal auf den Bildschirm schreiben kann.

- a) Wie viele Tassen Kaffee werden hergegeben?
- b) Welche Tasse(n) Kaffee kann Anna potentiell trinken?
- c) Welche Tasse(n) Kaffee kann Berta potentiell trinken?
- d) Welche Tasse(n) kann Berta potentiell trinken, wenn Anna die 3. Tasse verschüttet?
- e) Als wievielte Zeile(n) kann der Text, der über die 1. Tasse Auskunft gibt, am Bildschirm erscheinen?
- f) Wie viele unterschiedliche Ausgabefolgen gibt es, bei denen Berta die 2. Tasse vergisst?
- g) Nehmen Sie an, die Methode `gib()` sei *nicht* `synchronized`. Geben Sie eine mögliche Ausgabe an, die vorher nicht möglich gewesen wäre.

Aufgabe 4 (Schreibtischlauf)

(9 Punkte)

Gegeben ist folgendes paralleles Java-Programm:

```
public class Buch extends Thread {
    static int kapitel, sektion;
    static synchronized void init(int kapitel, int sektion) {
        Buch.kapitel = kapitel; Buch.sektion = sektion;
    }
    static synchronized void neueSektion() { sektion++; }
    static synchronized String print() {
        return kapitel + "." + sektion;
    }
    Buch(int kapitel, int sektion) { init(kapitel, sektion); }
    public void run() {
        System.out.println("Abschnitt:_" + print());
        neueSektion();
        System.out.println("Folgeabschnitt:_" + print());
    }
    public static void main(String[] args) {
        new Buch(1, 5).start(); new Buch(20, 10).start();
    }
}
```

Hinweis: Konstruktoren von **Thread**-Objekten werden vollständig abgearbeitet, bevor mittels **start** der Thread zu arbeiten beginnt.

- a) Wie oft wird die Methode **print()** insgesamt von beiden **Threads** aufgerufen?
- b) Welche unterschiedlichen Werte könnte **print()** ausgeben?
(Es geht nur um die Werte, nicht um die Reihenfolge in der Ausgabe!)

als Abschnitt	als Folgeabschnitt

Aufgabe 4 (Schreibtischlauf)

(7 Punkte)

Gegeben ist folgendes parallele Java-Programm:

```
public class Rhyme extends Thread {
    final Object a, b;
    public Rhyme(Object a, Object b) {
        this.a = a; this.b = b;
    }
    public void run() {
        System.out.println("Messer");
        synchronized (a) {
            System.out.println("Gabel");
            synchronized (b) {
                System.out.println("Schere");
            }
            System.out.println("Licht");
        }
    }
}
public static void main(String[] args) throws Exception {
    Object o1 = new Object();
    Object o2 = new Object();
    Thread t1 = new Rhyme(o1, o2);
    Thread t2 = new Rhyme(o2, o1);
    t1.start(); t2.start();
    t1.join(10000); t2.join(10000);
    System.exit(0);
}
```

- a) Warum könnte das Programm weniger als 8 Zeilen Text ausgeben?

Wieviele Zeilen werden dann ausgegeben?

Was ist das letzte Wort, das dann ausgegeben wird?

- b) Welche unterschiedlichen Ausgaben, die keine 8 Zeilen lang sind, kann das Programm produzieren? Welche davon ist unter der Annahme einer zufälligen Ablaufplanung die unwahrscheinlichste bzw. seltenste?

Aufgabe 4 (Schreibtischlauf)

(9 Punkte)

Gegeben ist folgendes parallele Java-Programm:

```

public class Count {

    static java.util.concurrent.CyclicBarrier barrier
        = new java.util.concurrent.CyclicBarrier(2, new Runnable() {
            public void run() { System.out.println("/-/"); } });

    static void foo(int i) { try {
        System.out.println(i++);
        barrier.await();
        System.out.println(i++);
        barrier.await();
        System.out.println(i++);
    } catch (Exception e) {}}

    synchronized void bar(int i) { try {
        System.out.println(i++);
        barrier.await();
        System.out.println(i++);
    } catch (Exception e) {}}

    public static void main(String[] args) throws Exception {
        final Count count = new Count();
        final Thread t1 = new Thread() {
            public void run() { count.foo(1); };
        };
        final Thread t2 = new Thread() {
            public void run() { count.bar(10); };
        };
        final Thread t3 = new Thread() {
            public void run() { count.bar(15); };
        };
        t1.start(); t2.start(); t3.start();
        t1.join(); t2.join(); t3.join();
    }
}

```

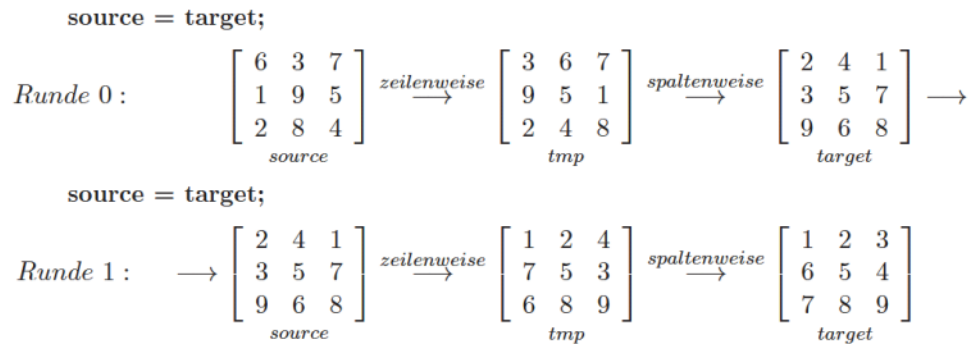
- Ignorieren Sie Ausnahmefälle und nehmen Sie an, dass nur ein Thread auf einmal auf den Bildschirm schreiben kann.
- Bei dieser Barriere schreibt der letzte betretende Thread `/-/` auf den Bildschirm. Insgesamt werden 2 Trennzeilen ausgegeben.
- Die Methode `meth2` ist **synchronized**!

	vor dem 1. <code>/-/</code>	zwischen den beiden <code>/-/</code>	nach dem 2. <code>/-/</code>
Anzahl ausgegebener Zeilen			
alle in verschiedenen Läufen möglichen ausgegebenen Zahlen			
Anzahl verschiedener möglicher Ausgaben			

Aufgabe 6 (Fehlersuche)

(12 Punkte)

Die `sort`-Methode der Java Klasse **SnakeSort** sortiert parallel ein übergebenes quadratisches `int`-Feld aufsteigend nach folgendem Schema:



Pro Zeile (bzw. pro Spalte) gibt es einen **WorkerThread**. In einer äußeren Schleife lässt der Main-Thread die Arbeiter so lange aktiv, bis sich das Feld nicht mehr ändert. Vor der nächsten Runde wird das teilweise sortierte Zwischenergebnis (**target**) in das Eingabefeld (**source**) kopiert. Die **WorkerThreads** sortieren die Zeilen des Eingabefelds (in **source**) abwechselnd auf- und absteigend und legen ihr Ergebnis in **tmp** ab. Nach einer Barriere sortieren die **WorkerThreads** die Spalten dieses Zwischenergebnisses aufsteigend und speichern die sortierten Spalten in **target**.

Ihre Aufgabe ist es, die **3 Fehler**, die bei der Parallelisierung des sequentiellen Codes entstanden sind, zu finden und **jeweils eine** mögliche Lösung vorzuschlagen.

- Hinweise:**
- Das Programm ist syntaktisch korrekt.
 - Alle 3 Fehler unterscheiden sich in ihrer Art und sind unabhängig zu lösen.
 - Es gibt keine Sichtbarkeits- und Synchronisationsproblematik bzgl. `glChanged`.
 - **WorkerThread** ist eine innere Klasse und sieht somit alle Instanz-Variablen.

	Zeile(n)	Fehlerbeschreibung	Lösung
Fehler 1			
Fehler 2			
Fehler 3			

```

1 import java.util.concurrent.CyclicBarrier;
2 public class SnakeSort {
3     CyclicBarrier mainBarrier, workBarrier;
4     int counter = 0;
5     boolean glChanged = false;
6     int [][] source, tmp, target;
7     final Object lock = new Object();
8
9     synchronized public int [][] sort(int [][] unsorted) throws Exception {
10         final int length = unsorted.length;
11         mainBarrier = new CyclicBarrier(length + 1);
12         workBarrier = new CyclicBarrier(length + 1);
13         createAndRunThreads(length);
14         target = unsorted;
15         do {
16             source = target;
17             target = new int[length][length];
18             tmp = new int[length][length];
19             mainBarrier.await();
20         } while (glChanged);
21         return target; // Wegen der Barriere ist ein join hier nicht noetig
22     }
23     class WorkerThread extends Thread {
24         public void run() {
25             try {
26                 do {
27                     mainBarrier.await();
28                     boolean changed = false;
29                     glChanged = false;
30                     int myLine = counter++;
31                     changed |= sortLine(source, tmp, myLine);
32                     workBarrier.await();
33                     int myColumn = --counter;
34                     changed |= sortColumn(tmp, target, myColumn);
35                     synchronized (lock) {
36                         glChanged |= changed;
37                     }
38                     mainBarrier.await();
39                 } while (glChanged);
40             } catch (Exception e) {}
41         }
42     }
43
44     // Folgende Methoden sind garantiert fehlerfrei:
45     // erzeugt und startet count WorkerThreads
46     public void createAndRunThreads(int count){...}
47     // sortiert je nach line abwechselnd auf- und absteigend
48     boolean sortLine(int [][] source, int [][] target, int line){...}
49     // sortiert die Spalte column aufsteigend
50     boolean sortColumn(int [][] source, int [][] target, int column){...}
51 }

```

Aufgabe 3 (Fehlersuche)

(12 Punkte)

Die **main**-Methode der Java-Klasse **Dinner** (linke Spalte) soll ein Abendessen unter **10** Philosophen (rechte Spalte) simulieren. In der Implementierung der Klasse **Dinner** befinden sich einige Fehler, die bei der Parallelisierung des sequentiellen Codes entstanden sind. In den Teilaufgaben a-c sollen Sie jeweils eine oder mehrere korrigierende Code-Zeilen angeben und dabei genau spezifizieren, welche Zeile(n) Ihr Code ersetzt. Sie dürfen die Klasse **Philosopher** **NICHT** ändern.

Hinweis: Das Programm ist syntaktisch korrekt. Der Code ist so formatiert, dass zusammengehörende Barrieren auf selber Höhe stehen (z.B. Zeile 35 und 94).

- a) Die erste Zeile der Ausgabe sollte **“10 von 10 Gaesten sind angekommen!”** (Zeilen 33/34) sein. Dies ist aber nicht bei jedem Durchlauf sichergestellt. Im Fehlerfall tritt z.B. die Ausgabe **“7 von 10 Gaesten sind angekommen!”** auf. Beheben Sie die Ursache des Problems, das durch den parallelen Ablauf entstanden ist.

Zeile(n)	Korrigierender Ersatz-Code

- b) Bei einigen Test-Durchläufen kommt es zu einer Verklemmung und die Ausgabe **“Alle Gaeste haben gegessen!”** (Zeile 44) erscheint nie.

Zeile(n)	Korrigierender Ersatz-Code

- c) Nachdem der Fehler in b) behoben wurde, kann es nach der Ausgabe von **“Alle Gaeste haben gegessen!”** (Zeile 44) zu einer weiteren Verklemmung kommen.

Zeile(n)	Korrigierender Ersatz-Code


```

1 import java.util.concurrent.*;
2 import java.util.concurrent.locks.*;
3
4 public class Dinner {
5     static int counter = 0;
6     static CyclicBarrier barrier =
7         new CyclicBarrier(11);
8     public static boolean[] paid =
9         new boolean[10];
10
11     public static void arrive(Object lock) {
12         synchronized(lock) {
13             counter++;
14         }
15     }
16
17     public static void main(String[] args)
18         throws Exception {
19         Philosopher[] guests =
20             new Philosopher[10];
21         ReentrantLock[] forks =
22             new ReentrantLock[10];
23         for (int i = 0; i < 10; i++) {
24             paid[i] = false;
25             forks[i] = new ReentrantLock();
26         }
27         for (int i = 0; i < 10; i++) {
28             guests[i] = new Philosopher(i,
29                 forks[i], forks[(i + 1) % 10]);
30             guests[i].start();
31         }
32         barrier.await();
33         out(counter + "_von_10_Gaesten_sind_"
34             + "angekommen!");
35         barrier.await();
36
37
38
39
40
41
42         barrier.await();
43         out("Alle_Gaeste_haben_gegessen!");
44         for (int i = 0; i < 10; i++) {
45             while (paid[i] != true) {}
46         }
47         barrier.await();
48         out("Alle_haben_bezahlt!");
49     }
50
51     public static void out(String s){
52         System.out.println(s);
53     }
54
55     public static void pay(int ID){
56         paid[ID]= true;
57     }
58 }
59
60 import java.util.concurrent.locks.*;
61
62
63 public class Philosopher extends Thread {
64     ReentrantLock leftFork, rightFork;
65     final int ID;
66
67
68     public Philosopher(int ID,
69         ReentrantLock leftFork,
70         ReentrantLock rightFork) {
71         this.ID = ID;
72         this.leftFork = leftFork;
73         this.rightFork = rightFork;
74     }
75
76     public void run() {
77         try {
78             int course = 0;
79             Dinner.arrive(this);
80
81
82
83
84
85
86
87
88
89
90
91             Dinner.barrier.await();
92
93
94             Dinner.barrier.await();
95             while (course < 5) {
96                 System.out.println("Denke...");
97                 leftFork.lock(); rightFork.lock();
98                 System.out.println("Esse...");
99                 rightFork.unlock(); leftFork.unlock();
100                course++;
101            }
102            Dinner.barrier.await();
103            Dinner.pay(ID);
104
105
106            Dinner.barrier.await();
107        } catch (Exception e) {
108        }
109    }
110 }
111
112
113
114
115
116
117
118 }

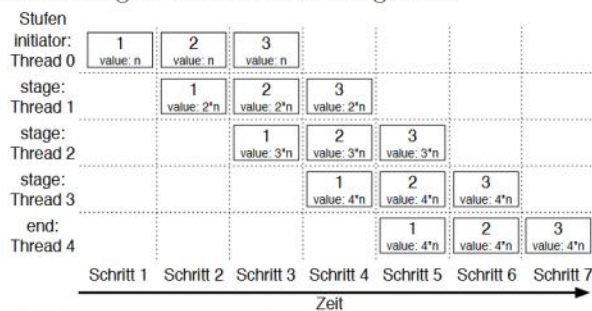
```


Aufgabe 4 (Fehlersuche)

(13 Punkte)

Die `main`-Methode der Java-Klasse `Desk` soll für eine übergebene Zahl n den Wert $12 * n$ berechnen. Zur Berechnung wird eine Pipeline mit fünf Stufen genutzt (siehe Abbildung). Die erste Stufe der Pipeline erzeugt drei Arbeitspakete, jeweils mit dem Wert `value = n`. Anschließend wird der Wert `value` durch die nächsten drei Stufen schrittweise erhöht. Die letzte Stufe der Pipeline überführt den Wert `value` in einen globalen Zähler.

Zwei aufeinanderfolgende Stufen der Pipeline synchronisieren sich mittels eines `Exchangers` und tauschen mit dessen Hilfe Objekte miteinander aus. Durch den Aufruf der `exchange`-Methode übergibt eine Stufe ein Objekt an einen `Exchanger` und wird blockiert. Sobald die Gegenseite ebenfalls die `exchange`-Methode aufruft, werden die Objekte übermittelt und die Verarbeitung in beiden Stufen fortgesetzt.



In der Implementierung der Klassen `Desk` und `Worker` befinden sich 4 Fehler, die den **korrekten** sowie **parallelen** Ablauf verhindern. Ihre Aufgabe ist es, diese Fehler zu finden und **jeweils eine** mögliche Lösung vorzuschlagen. Dabei sollen Sie jeweils eine oder mehrere fehlerbehebende Code-Zeilen angeben und genau spezifizieren, welche Zeile(n) Ihr Code ersetzt. **Hinweis:** Das Programm ist syntaktisch korrekt und Sie können davon ausgehen, dass keine Exceptions auftreten.

	Zeile(n)	Fehlerbeschreibung	Lösung
Fehler 1			
Fehler 2			
Fehler 3			
Fehler 4			

```

1 import java.util.concurrent.*;
2
3 public class Desk {
4     private static int n;
5     private static CyclicBarrier barrier = new CyclicBarrier(5);
6     private static int counter = 0;
7
8     public static void main(String[] args) throws Exception {
9         n = Integer.parseInt(args[0]);
10        Exchanger[] exchangers = new Exchanger[5];
11        for (int i = 0; i < 5; i++) {
12            exchangers[i] = new Exchanger<Integer>();
13        }
14
15        Worker[] threads = new Worker[5];
16        threads[0] = new Worker(0);
17        for (int i = 1; i < 5; i++) {
18            threads[i] = new Worker(i);
19            threads[i].exOut = exchangers[i];
20            threads[i].exIn = exchangers[i];
21        }
22
23        for (int i = 0; i < 5; i++) {
24            threads[i].run();
25        }
26
27        System.out.println("Input:_" + n + "_Result:_" + 12 * n + " = " + counter);
28        barrier.await();
29    }
30
31    static class Worker extends Thread {
32        public final int id;
33        public Exchanger<Integer> exIn, exOut;
34
35        public Worker(int id) {
36            this.id = id;
37        }
38        private void initiator() throws Exception {
39            for (int i = 0; i < 3; i++) {
40                this.exOut.exchange(n);
41            }
42            barrier.await();
43        }
44        private void stage() throws Exception {
45            for (int i = 0; i < 3; i++) {
46                int value = this.exIn.exchange(-1); //dummy value
47                this.exOut.exchange(value + n);
48            }
49        }
50        private void end() throws Exception {
51            for (int i = 0; i < 3; i++) {
52                counter += this.exIn.exchange(-1); //dummy value
53            }
54            barrier.await();
55        }
56        public void run() {
57            try {
58                if (this.id == 0) initiator();
59                else if (this.id == 4) end();
60                else stage();
61            } catch (Exception e) {
62            }
63        } // END run()
64    } // END class Worker
65 } // END class Desk
66

```

Aufgabe 6 (Sichtbarkeit und Wettlaufsituation)

(13 Punkte)

Mit der Klasse **RandomNumGen** wird durch den Aufruf von **getDigits** ein Array von 6 Pseudozufallsziffern erzeugt. Dazu werden Sichtbarkeitseffekte und Wettlaufsituationen ausgenutzt. Die Ziffern werden in den Variablen **d0**, **d1**, **d3** und in den **volatile** Variablen **dv2**, **dv4**, **dv5** vorberechnet und danach in dem Ergebnis-Array **digits** gespeichert. Beispielsweise wird **dv2** zur Berechnung von **digits[2]** verwendet. Ihre Aufgabe in den einzelnen Teilaufgaben ist es, die Eigenschaften der Zufallsziffern zu beschreiben.

Hinweis: Das Programm ist syntaktisch korrekt und Sie können davon ausgehen, dass es terminiert und keine Exceptions auftreten.

```

1 import java.util.concurrent.CyclicBarrier;
2
3
4 public class RandomNumGen {
5     int d0, d1, d3;
6     volatile int dv2, dv4, dv5;
7     CyclicBarrier barrier =
8         new CyclicBarrier(2);
9     int[] digits = new int[6];
10
11
12
13
14     public class Thread1 extends Thread {
15         public void run() {
16             try {
17
18                 barrier.await();
19
20                 d0++;
21                 d1++;
22
23                 dv2++;
24
25                 d3 = 9;
26
27                 barrier.await();
28
29
30                 for (int i = 0; i < 4; i++) {
31                     dv4++;
32                 }
33
34                 barrier.await();
35
36
37                 barrier.await();
38
39                 dv5 = 7;
40             } catch (Exception e) { }
41         } //END OF run
42     } //END OF Thread1
43
44
45
46     public int[] getDigits() {
47         d0 = 0; d1 = 0; d3 = 0;
48         dv2 = 0; dv4 = 0; dv5 = 0;
49         Thread t1 = new Thread1();
50         Thread t2 = new Thread2();
51         t1.start(); t2.start();
52         try {
53             t1.join(); t2.join();
54             digits[5] = dv5;
55         } catch (Exception e) { }
56         return digits;
57     }
58
59
60     public class Thread2 extends Thread {
61         public void run() {
62             try {
63
64                 barrier.await();
65
66                 digits[1] = d1;
67                 digits[0] = d0;
68
69                 digits[2] = dv2;
70
71                 barrier.await();
72
73                 digits[3] = d3;
74
75                 for (int i = 0; i < 4; i++) {
76                     dv4++;
77                 }
78
79                 barrier.await();
80
81                 digits[4] = dv4;
82
83                 barrier.await();
84
85                 dv5 = 5;
86             } catch (Exception e) { }
87         } //END OF run
88     } //END OF Thread2
89 } //END OF RandomNumGen

```

- a) Wie viele Kombinationen aus **digits[0]** und **digits[1]** können durch den Programmablauf entstehen? Begründen Sie Ihre Antwort.

- b) Welche Werte kann **digits[2]** annehmen? Begründen Sie Ihre Antwort.

- c) Welche Werte kann **digits[3]** annehmen? Begründen Sie Ihre Antwort.

- d) Welche Werte kann **digits[4]** annehmen? Begründen Sie Ihre Antwort.

- e) Welche Werte kann **digits[5]** annehmen? Begründen Sie Ihre Antwort.

Aufgabe 4 (Barrieren)

(12 Punkte)

Gegeben ist das folgende parallele Java-Programm:

```

1  import java.util.concurrent.*;
2
3  public class BarrierTest {
4      public static volatile int a = 0, b = 0;
5      public static CyclicBarrier barrier =
6          new CyclicBarrier(4);
7      public static CountDownLatch latch =
8          new CountDownLatch(3);
9      public static Object lock = BarrierTest.class;
10
11     public static class MyThreadA extends Thread {
12         public void run() {
13             try {
14                 synchronized (lock) {
15                     a++;
16                     System.out.println("a=_ " + a);
17                 }
18                 barrier.await();
19                 if (b == 0) {
20                     synchronized (lock) {
21                         if (b == 0) {
22                             b = 1;
23                         }
24                     }
25                     System.out.println("b=_ " + b);
26                 }
27                 latch.countDown();
28             } catch (Exception e) { }
29         }
30     }
31
32     public static void main(String[] args)
33         throws Exception {
34         Thread tA_1 = new MyThreadA();
35         Thread tA_2 = new MyThreadA();
36         Thread tB_3 = new MyThreadB();
37         Thread tB_4 = new MyThreadB();
38         tA_1.start();
39         tA_2.start();
40         tB_3.start();
41         while (a < 3) { }
42         System.out.println("Thread_1,_2,_3_started.");
43         tB_4.start();
44         latch.await();
45         System.out.println("Thread_4_started.");
46         tA_1.join();
47         tA_2.join();
48         tB_3.join();
49         tB_4.join();
50     }
51
52     public static class MyThreadB extends Thread {
53         public void run() {
54             try {
55                 synchronized (BarrierTest.class) {
56                     a++;
57                     System.out.println("a=_ " + a);
58                 }
59                 barrier.await();
60                 if (b == 0) {
61                     synchronized (lock) {
62                         if (b == 0) {
63                             b = 2;
64                         }
65                     }
66                     System.out.println("b=_ " + b);
67                 }
68                 latch.countDown();
69             } catch (Exception e) { }
70         }
71     }
72 } //END OF BARRIERTEST

```

Hinweis: Das Programm ist syntaktisch korrekt und Sie können davon ausgehen, dass weder Exceptions noch Deadlocks auftreten.

a) Welcher Text steht in der ersten Zeile der Ausgabe? Begründen Sie Ihre Antwort.

b) In welchen Zeilen der Ausgabe kann "Thread 1, 2, 3 started." stehen? Begründen Sie Ihre Antwort.

c) Wie oft wird **b** ausgegeben? Begründen Sie Ihre Antwort.

d) Die Ausgabe "Thread 4 started." steht in welcher Zeile?

☐

Letzte Zeile.

☐

Vorletzte oder letzte Zeile.

☐

Vorletzte Zeile.

☐

Beliebige Zeile.

Begründung:

Aufgabe 4 (Locks)

(12 Punkte)

Gegeben ist das unten stehende, syntaktisch korrekte parallele Java-Programm.

Sie können von Folgendem ausgehen: höchstens in den Methoden `lock()` und `unlock()` kann eine Exception (`IllegalMonitorStateException`) auftreten; der Betrag eines jeden Accounts wird niemals negativ; das Hauptprogramm startet mindestens zwei Threads vom Typ `MyThread` mit jeweils dem selben Bank-Objekt; und zu jedem Zeitpunkt können die Methoden `transfer()`, `checkTotalMoney()`, `withdraw()` nebenläufig ausgeführt werden.

```

1  class SimpleLock {
2      private volatile Thread owner = null;
3
4      public void lock() throws
5      IllegalMonitorStateException {
6          Thread self = Thread.currentThread();
7          if (owner == self) {
8              throw new IllegalMonitorStateException();
9          }
10         while (true) {
11             if (owner == null) {
12                 owner = self;
13                 return;
14             }
15         }
16     }
17
18     public void unlock() throws
19     IllegalMonitorStateException {
20         if (owner != Thread.currentThread()) {
21             throw new IllegalMonitorStateException();
22         }
23         owner = null;
24     }
25 }
26
27 class Account {
28     public final SimpleLock L = new SimpleLock();
29     public int money = 1000;
30 }
31
32 class Bank {
33     private Account[] accounts;
34     public final SimpleLock L = new SimpleLock();
35     public int totalMoney = 0;
36     public final int numAccounts = 4096;
37
38     public Bank() {
39         totalMoney = numAccounts * 1000;
40         accounts = new Account[numAccounts];
41         for (int i = 0; i < numAccounts; i++) {
42             accounts[i] = new Account();
43         }
44     }
45
46     public Account getAccount(int i) {
47         return accounts[i];
48     }
49 }
50
51 class MyThread extends Thread {
52     private final Bank B;
53
54     private void transfer(int f,
55                          int t, int amount) {
56         Account from = B.getAccount(f);
57         Account to = B.getAccount(t);
58
59         from.L.lock();
60         to.L.lock();
61
62         from.money -= amount;
63         to.money += amount;
64
65         from.L.unlock();
66         to.L.unlock();
67     }
68
69     private void checkTotalMoney() {
70         B.L.lock();
71         int total = B.totalMoney;
72         B.L.unlock();
73
74         int seen = 0;
75
76         int n = B.numAccounts;
77         for (int i = 0; i < n; i++) {
78             Account a = B.getAccount(i);
79             a.L.lock();
80             seen += a.money;
81             a.L.unlock();
82         }
83
84         if (total != seen) {
85             System.err.println("error");
86         }
87     }
88
89     private void withdraw(int i,
90                          int amount) {
91         Account a = B.getAccount(i);
92
93         B.L.lock();
94         a.L.lock();
95
96         a.money -= amount;
97         B.totalMoney -= amount;
98
99         a.L.unlock();
100        B.L.unlock();
101    }
102
103    MyThread(Bank B) { this.B = B; }
104    public void run() { ... }
105 }
106

```

- a) Begründen Sie, weshalb die Implementierung von `SimpleLock` nicht thread-sicher ist.
- b) Auch wenn nur ein einziger Thread aktiv ist und die Methode `transfer()` (Zeile 54) aufruft, kann es zu einer `IllegalMonitorStateException` kommen. In welcher Situation ist dies möglich?
- c) Nehmen Sie an, dass `SimpleLock` thread-sicher ist. Begründen Sie, welcher Fehler auftreten kann, wenn zwei Threads gleichzeitig `transfer()` aufrufen? Beschreiben Sie, wie sich der Fehler beheben lässt.
- d) Es soll die Invariante gelten, dass der in `totalMoney` (Zeile 35) gespeicherte Wert der Summe aller Kontostände entspricht. Nehmen Sie an, dass `SimpleLock`, sowie `Bank` und `MyThread` thread-sicher sind. Nur `checkTotalMoney()` (Zeile 69) ist defekt. Es kann zur Laufzeit dazu kommen, dass ein Thread in der Methode `checkTotalMoney()` „error“ (Zeile 85) ausgibt, wenn zeitgleich ein anderer Thread `withdraw()` (Zeile 89) aufruft. Beschreiben Sie, in genau welcher Konstellation der Fehler auftritt und wie Sie `checkTotalMoney()` reparieren können.

Aufgabe 5 (Schreibtischlauf)

(11 Punkte)

Gegeben ist das unten stehende, syntaktisch korrekte, parallele Java-Programm.
Sie können davon ausgehen, dass keine Exceptions auftreten.

```

1  import java.util.concurrent.*;
2  import java.util.concurrent.locks.*;
3
4  public class Bar {
5      CyclicBarrier barrier = new CyclicBarrier(3);
6      ReentrantLock lock = new ReentrantLock();
7      int counter = 0;
8      ConcurrentHashMap<Integer, String> map =
9          new ConcurrentHashMap<>();
10
11
12      public class Thread1 extends Thread {
13          public void run() {
14              try {
15                  barrier.await();
16                  counter++;
17                  barrier.await();
18
19
20                  lock.lock();
21                  if (map.isEmpty()) {
22                      map.put(0, "42");
23                  } else {
24                      map.put(0, "of_Life");
25                  }
26                  lock.unlock();
27                  synchronized(this) {
28                      String value = map.get(1);
29                      if (value == null) {
30                          map.put(1, "Adams");
31                      } else {
32                          map.put(1, "The_Universe");
33                      }
34                  }
35                  barrier.await();
36              } catch (Exception e) { }
37          }
38      }
39
40      public void execute() {
41          System.out.println("Question_to_Deep_Thought:");
42          System.out.println("");
43          Thread1 thread1 = new Thread1();
44          Thread2 thread2 = new Thread2();
45          thread1.start();
46          thread2.start();
47          try {
48              System.out.println("Answer_to_the");
49              barrier.await();
50              barrier.await();
51              barrier.await();
52              System.out.println(map.get(0));
53              System.out.println(map.get(1));
54              thread1.join();
55              thread2.join();
56              System.out.println("and_Everything");
57          } catch (Exception e) { }
58      }
59
60      public static void main(String[] args) {
61          Bar foo = new Bar();
62          foo.execute();
63      }
64
65      public class Thread2 extends Thread {
66          public void run() {
67              try {
68                  barrier.await();
69                  counter++;
70                  barrier.await();
71
72                  if (counter == 2) {
73                      System.out.println("Ultimate_Question");
74                  } else {
75                      System.out.println("Douglas");
76                  }
77                  lock.lock();
78                  if (map.isEmpty()) {
79                      map.put(0, "42");
80                  } else {
81                      map.put(0, "of_Life");
82                  }
83                  lock.unlock();
84                  synchronized(this) {
85                      String value = map.get(1);
86                      if (value == null) {
87                          map.put(1, "Adams");
88                      } else {
89                          map.put(1, "The_Universe");
90                      }
91                  }
92                  barrier.await();
93              } catch (Exception e) { }
94          }
95      }

```

```

1  Question to Deep Thought:
2
3  Answer to the
4  Ultimate Question
5  of Life
6  The Universe
7  and Everything

```

Abbildung 1: Ausgabe von Aufgabe 4b).

- a) Tragen Sie die Ausgabe des Programms in die folgende Tabelle ein. Durch die nebenläufige Ausführung des Programms kann es in einigen Zeilen zu alternativen Ausgaben kommen. Geben Sie diese in der entsprechenden Spalte an.

Ausgabe	Alternative Ausgaben
Question to Deep Thought:	<i>keine Alternative</i>
Leerzeile	<i>keine Alternative</i>
and Everything	<i>keine Alternative</i>

- b) Beheben Sie nun die Nebenläufigkeitsprobleme des Programms so, dass es garantiert den Text aus Abbildung 1 ausgibt. Außerdem dürfen Sie nur Barrieren, Locks, `synchronized`, `volatile`, oder atomare Datentypen zur Behebung der Probleme verwenden. Insbesondere dürfen dabei die Methoden `execute`, `main`, die Einträge der `HashMap` und die Textausgaben nicht verändert werden. Sie dürfen zur Behebung der Probleme nur Zeilen ersetzen und gegen eine oder mehrere Zeilen Java-Code austauschen. Geben Sie zu jeder Ersetzung die ersetzten Zeilen an. Achten Sie darauf, dass ihre Änderungen zu korrektem Java Code führen.

Sinnlose Beispiele:

```
Zeile 54+55: barrier.await();
Zeile 20: barrier.await(); lock.lock();
Zeile 2: CyclicBarrier barrier = new CyclicBarrier(4);
```

Aufgabe 3 (Fehlersuche)

(12 Punkte)

In dieser Aufgabe sind verschiedene kleine Programme gegeben. Beschreiben Sie **kurz** wieso die Ausführung von `execute` manchmal nicht terminiert. Beheben Sie danach den jeweiligen Fehler. Sie dürfen zur Behebung der Probleme nur Zeilen ersetzen und gegen eine oder mehrere Zeilen Java-Code austauschen. Geben Sie zu jeder Ersetzung an, welche Zeilennummer Ihre Code-Zeile ersetzen soll. Achten Sie darauf, dass ihre Änderungen zu korrektem Java-Code führen. **Der Code von `execute` darf nicht verändert werden.**

a)

```
1 public class Error1 {
2     static boolean finished = false;
3     static class MyThread extends Thread {
4         public void run() {
5             while (!finished) { }
6         }
7     }
8
9     public static void execute() throws InterruptedException {
10         MyThread thread = new MyThread();
11         thread.start();
12         finished = true;
13         thread.join();
14     }
15 }
```

Stichwortartige Problembeschreibung:

Beheben Sie den Fehler:

b)

```
1 import java.util.concurrent.CountDownLatch;
2
3 public class Error2 {
4     static CountDownLatch latch = new CountDownLatch(2);
5     static class MyThread extends Thread{
6         public void run() {
7             if (latch.getCount() == 2) {
8                 latch.countDown();
9             }
10        }
11    }
12
13    public static void execute() throws InterruptedException {
14        MyThread thread1 = new MyThread();
15        thread1.start();
16        MyThread thread2 = new MyThread();
17        thread2.start();
18        while (latch.getCount() != 1){ Thread.sleep(1000); }
19    }
20 }
```

Stichwortartige Problembeschreibung:

Beheben Sie den Fehler:

c)

```
1  import java.util.concurrent.CyclicBarrier;
2
3  public class Error3 {
4      static CyclicBarrier barrier = new CyclicBarrier(3);
5      static class MyThread extends Thread {
6          public void run() {
7              try {
8                  barrier.await();
9              } catch (Exception e) { }
10         }
11     }
12
13     public static MyThread foo() {
14         MyThread thread2 = new MyThread();
15         thread2.run();
16         return thread2;
17     }
18
19     public static void execute() throws Exception {
20         MyThread thread1 = new MyThread();
21         thread1.start();
22         MyThread thread2 = foo();
23         barrier.await();
24         thread1.join(); thread2.join();
25     }
26 }
```

Stichwortartige Problembeschreibung:

Beheben Sie den Fehler:

Aufgabe 2 (Schreibtischlauf)

(13 Punkte)

In dieser Aufgabe sind drei kleine Programme gegeben. Geben Sie jeweils an, welche Ausgaben bei der Ausführung von `execute` möglich sind. Die Reihenfolge der ausgegebenen Zeilen soll dabei ignoriert werden. Begründen Sie **kurz** ihre Antwort.

Hinweis: Die Methode `execute` ist in allen drei Teilaufgaben identisch.

a)

```

1  public class Run1 {
2      int x = 0; int y = 0;
3
4      class MyThread1 extends Thread{
5          public void run() {
6              x = 1;
7              System.out.println("y=_"+y);
8          }
9      }
10
11     class MyThread2 extends Thread{
12         public void run() {
13             y = 1;
14             System.out.println("x=_"+x);
15         }
16     }
17
18     public void output() {
19         // nothing to print here
20     }
21
22     public void execute() throws InterruptedException {
23         MyThread1 thread1 = new MyThread1();
24         MyThread2 thread2 = new MyThread2();
25         thread1.start(); thread2.start();
26         thread1.join(); thread2.join();
27         output();
28     }
29 }

```

Ausgaben:

	Ausgabe (MyThread1)	Ausgabe (MyThread2)
Möglichkeit 1	y =	x =
Möglichkeit 2	y =	
...		
...		
...		

Begründung:

b)

```

1  public class Run2 {
2      Object lock = new Object();
3      int counter = 0;
4
5      class MyThread1 extends Thread{
6          public void run() {
7              Object myLock = lock;
8              for (int i = 0; i < 2; i++) {
9                  synchronized(myLock) {
10                     counter++;
11                     myLock = new Object();
12                 }
13             }
14         }
15     }
16
17     class MyThread2 extends Thread{
18         public void run() {
19             synchronized(lock) {
20                 counter++;
21             }
22         }
23     }
24
25     public void output() {
26         System.out.println("counter_=" + counter);
27     }
28
29     public void execute() throws InterruptedException {
30         // Identical to execute in exercise a)!
31         MyThread1 thread1 = new MyThread1();
32         MyThread2 thread2 = new MyThread2();
33         thread1.start(); thread2.start();
34         thread1.join(); thread2.join();
35         output();
36     }
37 }

```

Ausgaben:

	Ausgabe (Main-Thread)
Möglichkeit 1	counter =
...	
...	
...	
...	

Begründung:

c)

```

1  public class Run3 {
2      BooleanWrapper wrapper = new BooleanWrapper();
3
4      class BooleanWrapper {
5          private Boolean value = false;
6          public synchronized void set(Boolean value) {
7              this.value = value;
8          }
9
10         public Boolean get() {
11             return value;
12         }
13     }
14
15     class MyThread1 extends Thread{
16         public void run() {
17             System.out.print("T1_=");
18             wrapper.set(true);
19             System.out.println("done.");
20         }
21     }
22
23     class MyThread2 extends Thread{
24         public void run() {
25             System.out.print("T2_=");
26             while(!wrapper.get()) { }
27             System.out.println("done.");
28         }
29     }
30
31     public void output() {
32         // nothing to print here
33     }
34
35     public void execute() throws InterruptedException {
36         // Identical to execute in exercise a)!
37         MyThread1 thread1 = new MyThread1();
38         MyThread2 thread2 = new MyThread2();
39         thread1.start(); thread2.start();
40         thread1.join(); thread2.join();
41         output();
42     }
43 }

```

Ausgaben:

	Ausgabe (MyThread1)	Ausgabe (MyThread2)
Möglichkeit 1	T1 =	T2 =
...		
...		
...		
...		

Begründung:

Aufgabe 4 (Synchronisation)

(12 Punkte)

In dieser Aufgabe sind zwei parallele Programme gegeben, deren Ausgabeverhalten (Rückgabewert, System.out) deterministisch ist. Ihre Aufgabe ist es, jeweils unnötige Synchronisationsoperationen zu entfernen. Sie können davon ausgehen, dass die execute-Methode nur von einem Thread aufgerufen wird.

a) Gegeben sei folgende Java-Klasse:

```

1 import java.util.concurrent.atomic.*;
2 public class SyncCount {
3     private volatile int globalHits;
4     private AtomicInteger todo;
5
6     public double execute()
7         throws InterruptedException {
8         final int total = 4711;
9
10        this.globalHits = 0;
11        this.todo =
12            new AtomicInteger(total);
13
14        Thread1 t1 = new Thread1();
15        Thread1 t2 = new Thread1();
16        t1.start();
17        t2.start();
18        t1.join();
19        t2.join();
20
21        return (4.0 * globalHits) / total;
22    }
23
24    private class Thread1 extends Thread {
25        private int localHits = 0;
26
27        private boolean continueLoop() {
28            synchronized (todo) {
29                return todo.getAndDecrement() > 0;
30            } // END OF continueLoop
31
32        public void run() {
33            while (continueLoop()) {
34                synchronized (this) {
35                    localHits += 1;
36                }
37            }
38            synchronized (SyncCount.class) {
39                globalHits += localHits;
40            }
41        } // END OF run
42    } // END OF Thread1
43 } // END OF SyncCount
44
```

Welche synchronized-Blöcke können entfernt werden, ohne dass sich dadurch der Rückgabewert der execute-Methode (Zeile 21) verändern kann?

Beginn	Ist ein Entfernen möglich? Begründen Sie Ihre Antwort kurz!
Zeile 27	
Zeile 34	
Zeile 38	

b) Gegeben sei folgende Java-Klasse:

```

1 import java.util.concurrent.*;
2 public class Barrier {
3     private CyclicBarrier barrier =
4         new CyclicBarrier(2);
5
6     private int i;
7     private int j;
8     private int k;
9     private volatile Integer v;
10
11     private class Thread2 extends Thread {
12         public void run() {
13             try {
14                 i = 42;
15
16                 barrier.await();
17                 k = -1;
18
19                 barrier.await();
20                 while (v == null) { /* Spin */ }
21                 k = Math.abs(i + j + v);
22                 barrier.await();
23                 // k >= 0 holds here
24
25             } catch (Exception e) { }
26         } // END OF run
27     } // END OF Thread2
28
31     public void execute()
32         throws InterruptedException {
33         Thread2 t2 = new Thread2();
34         Thread3 t3 = new Thread3();
35         t2.start();
36         t3.start();
37         t2.join();
38         t3.join();
39     }
40
41     private class Thread3 extends Thread {
42         public void run() {
43             try {
44                 v = null;
45
46                 barrier.await();
47                 j = 16 - i / 2;
48                 v = new Integer(5);
49                 barrier.await();
50
51             } catch (Exception e) { }
52         } // END OF run
53     } // END OF Thread3
54     } // END OF Barrier

```

Welche Paare von `barrier.await()`-Aufrufen können entfernt werden, ohne dass sich dadurch der bei einem Aufruf von `execute` auf dem Bildschirm ausgegebene Wert (Zeile 56) verändern kann?

Zeilen	Ist ein Entfernen möglich? Begründen Sie Ihre Antwort kurz!
16/46	
19/49	
22/52	

Aufgabe 4 (Schreibtischlauf)

(12 Punkte)

In dieser Aufgabe ist in jeder Teilaufgabe jeweils ein kleines Programm gegeben. Geben Sie jeweils an, wie viele verschiedene Ausgaben bei der Ausführung von `main` möglich sind. Beantworten Sie danach die Frage über die Ausgaben des Programms.

a)

```
1 public class Printer1 {
2
3   String[] s = new String[]{null};
4
5   class Thread1 extends Thread {
6     public void run() {
7       String tmp = s[0];
8       s[0] = s[1];
9       s[1] = tmp;
10    }
11  }
12
13  class Thread2 extends Thread {
14    public void run() {
15      String tmp = s[0];
16      s[0] = s[2];
17      s[2] = tmp;
18    }
19  }
20
21  public static void main(String[] args) {
22    Printer1 p = new Printer1();
23    Thread1 t1 = p.new Thread1();
24    Thread2 t2 = p.new Thread2();
25    p.s = new String[] {
26      "xkcd", "wtf", "omg"
27    };
28    t1.start(); t2.start();
29    System.out.println(p.s[0]);
30  }
31 }
```

Wie viele verschiedene Ausgaben sind möglich?

Ist `null` eine mögliche Ausgabe? Begründen Sie *kurz* Ihre Antwort.

b)

```
1 public class Printer2 {  
2  
3     String[] s = new String[] {"Geralt"};  
4     volatile boolean done = false;  
5  
6     class Thread1 extends Thread {  
7         public void run() {  
8             s[0] = "Triss";  
9             done = true;  
10        }  
11    }  
12  
13    class Thread2 extends Thread {  
14        public void run() {  
15            s[0] = "Yennefer";  
16            done = true;  
17        }  
18    }  
19  
20    public static void main(String[] args) {  
21        Printer2 p = new Printer2();  
22        Thread1 t1 = p.new Thread1();  
23        Thread2 t2 = p.new Thread2();  
24        t1.start(); t2.start();  
25        while (!p.done) { }  
26        System.out.println(p.s[0]);  
27    }  
28 }
```

Wie viele verschiedene Ausgaben sind möglich?

Ist **Geralt** eine mögliche Ausgabe? Begründen Sie *kurz* Ihre Antwort.

c)

```
1 public class Printer3 {
2
3     volatile String[] s = new String[] {
4         "doom", "doom"
5     };
6
7     class Thread1 extends Thread {
8         public void run() {
9             synchronized(this) {
10                 if (s[0].equals("doom")) {
11                     s[0] = "iddqd";
12                     s[1] = "iddqd";
13                 }
14             }
15         }
16     }
17
18     class Thread2 extends Thread {
19         public void run() {
20             synchronized(this) {
21                 if (s[0].equals("doom")) {
22                     s[0] = "42";
23                     s[1] = "42";
24                 }
25             }
26         }
27     }
28
29     public static void main(String[] args) {
30         Printer3 p = new Printer3();
31         Thread1 t1 = p.new Thread1();
32         Thread2 t2 = p.new Thread2();
33         t1.start(); t2.start();
34         try {
35             t1.join(); t2.join();
36         } catch (InterruptedException e) { }
37         System.out.println(
38             p.s[0] + " " + p.s[1]
39         );
40     }
41 }
```

Wie viele verschiedene Ausgaben sind möglich?

Ist **doom doom** eine mögliche Ausgabe? Begründen Sie *kurz* Ihre Antwort.

Aufgabe 4 (Schreibtischlauf)

(10 Punkte)

Gegeben sei folgendes paralleles Programm:

```

1 import java.util.concurrent.*;
2 public class NumberProg {
3     volatile int a = 1;
4
5     volatile int[] q = new int[] {0, 0};
6
7     volatile int x = 0;
8     int y = 0;
9
10    CyclicBarrier barrier
11        = new CyclicBarrier(2);
12
13    class T1 extends Thread {
14        public void run() {
15            try {
16                synchronized (barrier) {
17                    a *= 2;
18                }
19
20                q[0] = 5;
21
22                barrier.await();
23
24                int t = q[0];
25                q[0] = q[1];
26                q[1] = t;
27
28                barrier.await();
29
30                y = 11;
31                x = 47;
32            } catch (Exception e) { }
33        }
34    } // END OF T1
35
36    public static void main(
37        String[] args)
38        throws Exception {
39        new NumberProg().exec();
40    }
41
42    public void exec() throws Exception {
43        T1 t1 = new T1();
44        T2 t2 = new T2();
45        t1.start();
46        t2.start();
47        t1.join();
48        t2.join();
49    }
50
51    class T2 extends Thread {
52        public void run() {
53            try {
54                synchronized (T2.class) {
55                    a *= 2;
56                }
57
58                q[1] = 7;
59
60                barrier.await();
61
62                q[1] = 42;
63
64                barrier.await();
65
66                System.out.println("a=" + a);
67
68                System.out.println("q0=" + q[0]);
69                System.out.println("q1=" + q[1]);
70
71                System.out.println("x=" + x);
72                System.out.println("y=" + y);
73            } catch (Exception e) { }
74        }
75    } // END OF T2
76
77    } // END OF NumberProg

```

Hinweis: Sie können davon ausgehen, dass während der Ausführung keine Exceptions auftreten.

a) Welche ggf. verschiedenen Werte können für a in Zeile 71 ausgegeben werden?

-

b) Welche ggf. verschiedenen Werte können für $q[0]$ und $q[1]$ in den Zeilen 73/74 ausgegeben werden?

-

c) Ist $x = 47$, $y = 0$ eine mögliche Ausgabe in den Zeilen 76/77? Begründen Sie Ihre Antwort.

Aufgabe 2 (Fehlersuche)

(9 Punkte)

Es sind zwei syntaktisch korrekte Programme gegeben, die sich bei paralleler Ausführung fehlerhaft verhalten. Die Programmausführung beginnt mit der jeweiligen `exec`-Methode. Finden und beheben Sie jeweils das Problem, indem Sie **nur** den Code in der linken Spalte verändern.

a) Die Ausführung von `exec` soll „3“ oder „4“ ausgeben:

<pre> 1 class TB extends Thread { 2 Cnt ca, cb; 3 4 TB(Cnt ca, Cnt cb) { 5 this.ca = ca; 6 this.cb = cb; 7 } 8 9 public void run() { 10 synchronized (ca) { 11 synchronized (cb) { 12 ca.add(cb); 13 } 14 } 15 } 16 }</pre>	<i>Diese Spalte <u>nicht</u> verändern!</i> <pre> 17 class Cnt { 18 private int x; 19 Cnt(int x) { this.x = x; } 20 int getX() { return this.x; } 21 void add(Cnt c) { 22 this.x += c.x; 23 } 24 } 25 26 void exec() throws Exception { 27 Cnt foo = new Cnt(1); 28 Cnt bar = new Cnt(2); 29 Thread t1 = new TB(foo, bar); 30 Thread t2 = new TB(bar, foo); 31 t1.start(); t2.start(); 32 t1.join(); t2.join(); 33 System.out.println(foo.getX()); 34 }</pre>
--	--

Warum gibt es neben den zwei Ausgaben noch eine dritte Möglichkeit, wie sich das Programm verhalten kann?

Beheben Sie das beschriebene Problem. Geben Sie auch die veränderte(n) Zeilennummer(n) an. Verändern Sie **nicht** die `Cnt`-Klasse und **nicht** die `exec`-Methode.

b) Die Ausführung von `exec` soll „zz“ ausgeben:

```
1  boolean b = false;
2
3  void set(String val) {
4      if (this.b) {
5          return;
6      }
7      this.s = val;
8      this.b = true;
9  }
10 String s = "a";
11
12 class TC extends Thread {
13     public void run() {
14         set("z");
15         System.out.print(s);
16     }
17 }
18
19 void exec() throws Exception {
20     Thread t1 = new TC();
21     Thread t2 = new TC();
22     t1.start(); t2.start();
23     t1.join(); t2.join();
24     System.out.println();
25 }
```

Warum ist auch eine andere Ausgabe als „zz“ möglich?

Beheben Sie das beschriebene Problem. Geben Sie auch die veränderte(n) Zeilennummer(n) an. Verändern Sie **nicht** die `run`- und **nicht** die `exec`-Methode und **nicht** das Feld `s`.

Aufgabe 5 (Schreibtischlauf)

(10 Punkte)

Gegeben sei folgendes paralleles Programm. Die Ausführung beginnt in der Methode `exec()`.

```

1 import java.util.concurrent.atomic.*;
2 import java.util.concurrent.*;
3
4 public class IntQueue {
5     int[] data = new int[4];
6     int nextWrite = 0;
7     CountdownLatch latch = null;
8
9     public void exec() throws Exception {
10         TWrite w1 = new TWrite(1);
11         TWrite w3 = new TWrite(3);
12         w1.start(); w3.start();
13         latch = new CountdownLatch(2);
14         latch.await();
15         for (int i = 0; i < 4; i++) {
16             System.out.print(data[i]);
17         }
18         System.out.println();
19     }
20     void add(int x) {
21         data[nextWrite++] = x;
22     }
23
24     class TWrite extends Thread {
25         int begin;
26
27         public TWrite(int begin) {
28             this.begin = begin;
29         }
30
31         public void run() {
32             try {
33                 add(begin);
34                 add(begin + 1);
35                 while (latch == null) {}
36                 latch.countDown();
37             } catch (Exception e) {}
38         }
39     } // ENDE VON TWrite
40 } // ENDE VON IntQueue

```

Hinweis: Sie können davon ausgehen, dass während der Ausführung keine Exceptions auftreten. Das Programm soll die vier Zahlen 1 bis 4 ausgeben. Aufgrund von verschiedenen Fehlern funktioniert das jedoch nicht zuverlässig.

- a) In den meisten Fällen unterscheiden sich nur die Ausgabereihenfolgen der vier Zahlen. Geben Sie alle unterschiedlichen Ausgabereihenfolgen an, die auftreten können. Ausgaben, bei denen nicht genau die vier Zahlen 1 bis 4 vorkommen, sollen ignoriert werden.
- b) Ist sichergestellt, dass `exec` die in `data[]` eingetragenen Werte sicher sieht? Falls *ja*: Wodurch ist dies sichergestellt? Falls *nein*: Wie könnte man das Problem beheben?
- c) Welche Parallelitätsprobleme können sonst noch auftreten? Nennen Sie **zwei** verschiedene Probleme. Erläutern Sie für jedes Problem, wie und warum es entsteht, und zeigen Sie eine Möglichkeit, dieses Problem zu verhindern.

Problem 1:

Lösung 1:

Problem 2:

Lösung 2:

Aufgabe 4 (Schreibtischlauf)

(11 Punkte)

Gegeben sei folgendes paralleles Programm. Gehen Sie davon aus, dass der einzige zu Beginn vorhandene Aktivitätsfaden die Methode `exec` aufruft.

```

1 import java.util.concurrent.*;
2
3 public class BrokenExec {
4
5     private BlockingQueue<Runnable> queue
6         = new LinkedBlockingQueue<>();
7     private int numWorkers;
8     private volatile boolean isShutdown;
9
10    private static final Runnable POISON_PILL
11        = ...
12
13
14    private class Worker extends Thread {
15        public void run() {
16            Runnable next = null;
17            try {
18                while ((next = queue.take())
19                    != POISON_PILL) {
20                    next.run();
21                }
22            } catch (InterruptedException e) { }
23
24            if (--numWorkers == 0) {
25                System.out.println("Finished!");
26            }
27        }
28    } // ENDE class Worker
29
30
31    class SubmitTask implements Runnable {
32        public void run() {
33            try {
34                submit(new HelloTask());
35            } catch (RejectedExecutionException e) {
36                System.out.println("Rejected!");
37            }
38        }
39    } // ENDE class SubmitTask
40
41    class ShutdownTask implements Runnable {
42        public void run() {
43            System.out.println("Shutdown");
44            shutdown();
45        } // ENDE class ShutdownTask
46
47    class HelloTask implements Runnable {
48        public void run() {
49            System.out.println("Hello");
50        } // ENDE class HelloTask
51
52
53    public void shutdown() {
54        isShutdown = true;
55        final int active = numWorkers;
56        for (int i = 0; i < active; ++i) {
57            queue.offer(POISON_PILL);
58        }
59    }
60
61    public void submit(Runnable task) {
62        if (isShutdown) {
63            throw new RejectedExecutionException();
64        }
65        queue.offer(task);
66    }
67
68    public void exec() {
69        isShutdown = false;
70        numWorkers = 4;
71        for (int i = 0; i < numWorkers; ++i) {
72            new Worker().start();
73        }
74
75        System.out.println("Begin");
76        submit(new SubmitTask());
77        submit(new ShutdownTask());
78    }
79
80 } // ENDE class BrokenExec

```

a) Welchem Zweck dienen die Giftpillen im Programm?

- b) Erläutern Sie, in welchem Fall aufgrund welcher Wettlaufsituation im gegebenen Programm „Finished!“ in Zeile 25 **nicht** ausgegeben wird. Wie lässt sich dieses Problem beheben?
- c) Begründen Sie, warum die Ausgabe „Rejected!“ in Zeile 36 möglich ist.
- d) Erläutern Sie, in welchem Fall aufgrund welcher Wettlaufsituation im gegebenen Programm **weder** „Hello“ (Zeile 49) **noch** „Rejected!“ (Zeile 36) ausgegeben wird. Wie lässt sich dieses Problem beheben?

Aufgabe 3 (Fehlersuche)

(12 Punkte)

In dieser Aufgabe sind zwei parallele Programme (A und B) gegeben. Diese sind syntaktisch korrekt, enthalten aber verschiedene Fehler. Sie können davon ausgehen, dass die Ausführung in `execute` beginnt, `execute` nicht nebenläufig ausgeführt wird und keine Ausnahmen auftreten.

a) Gegeben ist folgendes Programm A:

```

1 import java.util.*;
2 public class MonteCarlo {
3
4     volatile boolean[] finished;
5     volatile long hits;
6
7     public double execute()
8         throws Exception {
9         finished = new boolean[4];
10        hits = 0;
11
12        for (int i = 0; i < 4; ++i) {
13            new TaskA(i).start();
14        }
15
16        for (int i = 0; i < 4; ++i) {
17            while (!finished[i]) {
18                Thread.sleep(1000);
19            }
20        }
21        // approx. pi
22        return hits / 1000000.0;
23    } // END OF execute
24
25    class TaskA extends Thread {
26        int id;
27        Random rng;
28
29        TaskA(int id) {
30            this.id = id;
31            this.rng = new Random(id);
32        }
33
34        public void run() {
35            for (int i = 0; i < 1000000; ++i) {
36                double x = rng.nextDouble();
37                double y = rng.nextDouble();
38                if (x * x + y * y <= 1) {
39                    synchronized (rng) {
40                        hits += 1;
41                    }
42                } // END OF for
43                finished[id] = true;
44            } // END OF run
45        }
46    }

```

Nennen und beschreiben Sie zwei Parallelisierungsfehler (inkl. Zeilennummer), die zu nicht-deterministischem Verhalten führen. Erklären Sie auch, wie/wo diese zu beheben sind.

Zeilennummer(n) Fehler 1:

Beschreibung Fehler 1:

Behebung Fehler 1:

Zeilennummer(n) Fehler 2:

Beschreibung Fehler 2:

Behebung Fehler 2:

b) Gegeben ist folgendes Programm B:

```

1 import java.util.concurrent.*;
2
3 public class GameOfLife {
4
5     CyclicBarrier barrier;
6     volatile boolean[] src;
7     volatile boolean[] dst;
8     int numGens;
9
10    public boolean[] execute()
11        throws Exception {
12        assert src.length == 32;
13        dst = new boolean[src.length];
14
15        barrier = new CyclicBarrier(4);
16        for (int i = 0; i < 4; ++i) {
17            new TaskB(i).start();
18        }
19        for (int g = 0; g < numGens;
20            ++g) {
21            barrier.await(); -----
22
23            boolean[] temp = src;
24            src = dst;
25            dst = temp;
26        }
27        return src;
28    }
29
30    static boolean nextGen(boolean[] a,
31        int i) { ... }
32
33    class TaskB extends Thread {
34        int id;
35
36        TaskB(int id) {
37            this.id = id;
38        }
39
40        public void run() {
41            try {
42                for (int g = 0; g < numGens; ++g) {
43                    // src.length == 32
44                    for (int i = 8 * id;
45                        i < 8 * (id + 1); ++i) {
46                        dst[i] = nextGen(src, i);
47                    }
48                }
49                barrier.await();
50            }
51            catch (Exception e) { }
52        }
53    }
54 }

```

Nennen und beschreiben Sie zwei Parallelisierungsfehler (inkl. Zeilennummer), die zu nicht-deterministischem Verhalten führen. Erklären Sie auch, wie/wo diese zu beheben sind.

Zeilennummer(n) Fehler 1:

Beschreibung Fehler 1:

Behebung Fehler 1:

Zeilennummer(n) Fehler 2:

Beschreibung Fehler 2:

Behebung Fehler 2:

Aufgabe 2 (Synchronisation)

(11 Punkte)

In dieser Aufgabe sind drei parallele Programme gegeben, deren Ausgabeverhalten (Rückgabewert, `System.out`) deterministisch ist. Ihre Aufgabe ist es, bestimmte unnötige Synchronisationsoperationen zu entfernen, ohne dabei das deterministische Verhalten zu ändern. Sie können davon ausgehen, dass die `execute`-Methode jeweils nur von einem Thread aufgerufen wird.

a) Gegeben ist folgendes Java-Programm:

```

1 public class SyncA {
2
3     boolean global;
4     volatile String[] str;
5
6
7     class T1 extends Thread {
8         public void run() {
9             while (true) {
10                 synchronized (SyncA.class) {
11                     if (global) break;
12                 }
13             }
14
15             synchronized (str) {
16                 str[0] = "Hallo";
17             }
18         }
19     } // END of class T1
20
21
22     public void execute() {
23         global = false;
24         str = new String[] { null };
25         new T1().start();
26         new T2().start();
27     }
28
29     class T2 extends Thread {
30         public void run() {
31             synchronized (SyncA.class) {
32                 global = true;
33             }
34
35             while (true) {
36                 synchronized (str) {
37                     if (str[0] != null) break;
38                 }
39                 System.out.print(str[0]);
40             }
41         } // END of class T2
42     }

```

Bearbeiten Sie die folgenden beiden Aufgaben unabhängig voneinander:

- i) Geben Sie eine begründete Einschätzung, ob der `synchronized`-Block in Zeile 10 entfernt werden kann:
- ii) Geben Sie eine begründete Einschätzung, ob die beiden `synchronized`-Blöcke in den Zeilen 15 und 35 entfernt werden können:

b) Gegeben ist folgendes Java-Programm:

```

1 import java.util.concurrent.  

2     atomic.*;  

3 public class SyncB {  

4     String msg;  

5     AtomicInteger c;  

6     AtomicBoolean b;  

7  

8     class T1 extends Thread {  

9         public void run() {  

10             c.addAndGet(2);  

11  

12             b.set(true);  

13  

14         }  

15     } // END of class T1  

16  

17     public void execute() {  

18         c = new AtomicInteger(0);  

19         b = new AtomicBoolean(false);  

20         new T1().start();  

21         new T2().start();  

22     }  

23  

24     class T2 extends Thread {  

25         public void run() {  

26             c.incrementAndGet();  

27  

28             while (!b.get()) { }  

29             System.out.println(c.get());  

30         }  

31     } // END of class T2  

32 }

```

Bearbeiten Sie die folgenden beiden Aufgaben unabhängig voneinander:

- i) Geben Sie eine begründete Einschätzung, ob der in Zeile 5 deklarierte `AtomicInteger` in einen `volatile int` überführt werden kann, wenn die Operationen in den Zeilen 10, 18, 26 und 29 entsprechend durch `int`-Operationen ersetzt werden:

Korrektorenhinweis: Auf alternative, richtige Lösungen achten, die zu Zeile 28 passen.

c) Gegeben ist folgendes Java-Programm:

```

1 import java.util.concurrent.*;
2 public class SyncC {
3     CyclicBarrier barrier;
4     String msg;
5     int v;
6     volatile boolean b;
7
8     class T1 extends Thread {
9         public void run() {
10             try {
11                 msg = "Ergebnis: ";
12                 barrier.await();
13                 -----
14                 while (v < 42) {
15                     v += 1;
16                 }
17                 b = true;
18                 barrier.await();
19                 -----
20             } catch (Exception e) { }
21         }
22     } // END of class T1
23 }
24
25
26
27 public void execute() {
28     barrier = new CyclicBarrier(2);
29     msg = null;
30     v = 0;
31     b = false;
32     new T1().start();
33     new T2().start();
34 }
35 class T2 extends Thread {
36     public void run() {
37         try {
38             -----
39             barrier.await();
40             -----
41             while (msg != null) { }
42             String tmp = msg;
43
44             -----
45             barrier.await();
46             -----
47             while (!b) { }
48             System.out.println(tmp + v);
49         } catch (Exception e) { }
50     }
51 } // END of class T2
52 }

```

Bearbeiten Sie die folgenden beiden Aufgaben unabhängig voneinander:

- i) Geben Sie eine begründete Einschätzung, ob die Barrieren in den Zeilen 13 und 39 entfernt werden können:

Aufgabe 3 (Schreibtischlauf)

(9 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass anfangs alle Instanzvariablen den initial zugewiesenen Wert enthalten und während der Programmausführung keine Ausnahmen auftreten.

a) Gegeben sei folgendes paralleles Programm:

```

1  public class Parallell {
2
3      String name = "foo";
4
5      Object lock = new Object();
6
7
8      class T1 extends Thread {
9          public void run() {
10             while (true) {
11                 synchronized (lock) {
12                     if (name.equals("foobar")) {
13                         break;
14                     }
15                 }
16             }
17         }
18     }
19
20     class T2 extends Thread {
21         public void run() {
22             synchronized (lock) {
23                 lock = new Object();
24                 name += "bar";
25             }
26         }
27     }
28
29     public void execute()
30         throws Exception {
31
32         T1 t1 = new T1();
33         T2 t2 = new T2();
34         t1.start(); t2.start();
35         t1.join(); t2.join();
36         System.out.println(name);
37     }
38 }

```

Erläutern und begründen Sie jeweils, ob es möglich ist, dass das Programm das angegebene Verhalten zeigen kann.

Verhalten	Erläuterung, ob/warum Verhalten möglich/nicht möglich ist
Ausgabe „foobar“	
Ausgabe „foo“	
Nichtterminierung	

b) Gegeben sei folgendes paralleles Programm:

```

1  import java.util.concurrent.*;
2  import java.util.concurrent.atomic.*;
3  public class Parallel2 {
4      CyclicBarrier barrier
5          = new CyclicBarrier(2);
6      AtomicInteger i
7          = new AtomicInteger();
8
9      class Task extends Thread {
10         public void run() {
11             try {
12                 i.getAndIncrement();
13                 barrier.await(); -----
14             } catch (Exception e) { }
15         }
16     }
17     public void execute()
18         throws Exception {
19
20         Task t1 = new Task();
21         Task t2 = new Task();
22         Task t3 = new Task();
23
24         t1.start();
25         t2.start();
26         t3.start();
27
28         -----
29         barrier.await();
30         System.out.println(i.get());
31     }
32 }

```

Erläutern und begründen Sie jeweils, ob es möglich ist, dass das Programm das angegebene Verhalten zeigen kann. Beziehen Sie sich dabei auf die Anzahl der Barrieren-Teilnehmer.

Verhalten	Erläuterung, ob/warum Verhalten möglich/nicht möglich ist
Ausgabe „3“	
Ausgabe „1“	
Nichtterminierung	

Aufgabe 4 (Schreibtischlauf)

(10 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird.

a) Gegeben sei folgendes paralleles Programm:

```

1 public class Parallell {
2
3     class ParallelSolver extends Thread {
4         public Solver s;
5         public void run() {
6             Solver s = new Solver(this);
7         }
8     }
9
10    class Solver {
11        public int value;
12        public Solver(ParallelSolver ps) {
13            value = 42;
14            ps.s = this;
15        }
16    }
17
18    public void execute()
19        throws Exception {
20        ParallelSolver ps =
21            new ParallelSolver();
22        ps.start();
23        System.out.println("the_answer:_" +
24            ps.s.value);
25    }
26 }

```

Erläutern und begründen Sie jeweils, ob es möglich ist, dass das Programm das angegebene Verhalten zeigen kann.

Verhalten	Erläuterung, ob/warum Verhalten möglich/nicht möglich ist
Ausgabe „the answer: 42“	
Ausgabe „the answer: 0“	
NullPointerException	

b) Gegeben sei folgendes paralleles Programm:

```

1 public class Parallel2 {
2     class T1 extends Thread {
3         public void run() {
4             a = 2;
5             b = 2;
6             c = 1;
7         }
8     }
9     class T2 extends Thread {
10        public void run() {
11            while(c == 0){
12                //waiting, do nothing
13            }
14            a++;
15            b++;
16        }
17    }
18    public int a, b, c;
19
20    public void execute()
21        throws Exception {
22        a = 0;
23        b = 0;
24        c = 0;
25        T1 t1 = new T1();
26        T2 t2 = new T2();
27        t1.start(), t2.start();
28        t1.join(), t2.join();
29        System.out.println("a:_" +
30            a + ",_b:_" + b);
31    }
32 }

```

Erläutern und begründen Sie jeweils, ob es möglich ist, dass das Programm das angegebene Verhalten zeigen kann.

Verhalten	Erläuterung, ob/warum Verhalten möglich/nicht möglich ist
Ausgabe „a: 4, b: 4“	
Ausgabe „a: 3, b: 3“	
Ausgabe „a: 1, b: 1“	
Ausgabe „a: 0, b: 0“	
Nichtterminierung	

Aufgabe 3 (Schreibtischlauf)

(10 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten und während der Programmausführung keine Ausnahmen auftreten.

a) Gegeben sei folgendes paralleles Programm:

```

1  import java.util.concurrent.*;
2
3  public class SL1 {
4
5      volatile int counter = 0;
6      volatile int result = 0;
7      CyclicBarrier barrier =
8          new CyclicBarrier(3);
9
10     ... class Task1 extends Thread {
11         ... synchronized void init1() {
12             counter++;
13         }
14
15         ... public void run() {
16             try {
17                 init1();
18                 barrier.await();
19                 result += counter;
20             } catch (Exception e) {}
21         }
22     }
23
24     ... class Task2 extends Thread {
25         ... synchronized void init2() {
26             counter++;
27         }
28
29         ... public void run() {
30             try {
31                 init2();
32                 barrier.await();
33                 result += 2 * counter;
34             } catch (Exception e) {}
35         }
36     }
37
38     ... void execute() {
39         try {
40             Task1 t1 = new Task1();
41             Task2 t2 = new Task2();
42             t1.start();
43             t2.start();
44             barrier.await();
45         } catch (Exception e) {}
46         System.out.println(
47             "c:_" + counter + "_" +
48             "r:_" + result);
49     }
50 }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie ihre Antwort, wo dies verlangt ist.

- i) Kann es bei der Berechnung von `counter` in den Zeilen 12 und 26 zu einer Wettlaufsituation kommen?

, weil

- ii) Kann es bei der Berechnung von `result` in den Zeilen 19 und 33 zu einer Wettlaufsituation kommen?

, weil

- iii) Kann `execute` am Ende nach `c`: den Wert 0 ausgeben (Z. 47)?

, weil

- iv) Was ist der kleinste Wert, den `execute` am Ende nach `r`: ausgeben kann (Z. 48)?

, weil

- v) Was ist der größte Wert, den das Programm nach `r`: ausgeben kann (Z. 48)?

b) Gegeben sei folgendes paralleles Programm:

```

1 public class SL2 {
2
3     class Con extends Thread {
4         int ownId;
5         volatile int nextId;
6         Con pending;
7
8         public Con(int ownId) {
9             this.ownId = ownId;
10        }
11
12        synchronized void
13            setNextId(int id) {
14                this.nextId = id;
15            }
16
17        synchronized void
18            processPending() {
19            if(pending == null) return;
20            pending.setNextId(ownId);
21            pending = null;
22        }
23
24        public void run() {
25            processPending();
26        }
27    }
28
29
30    public void execute() {
31        Con c1 = new Con(42);
32        Con c2 = new Con(1337);
33        c1.pending = c2; c2.pending = c1;
34        c1.start(); c2.start();
35        try {
36            c1.join(); c2.join();
37        } catch (Exception e) {}
38        System.out.println("c1:_ " + c1.ownId +
39            "_->_" + c1.nextId);
40        System.out.println("c2:_ " + c2.ownId +
41            "_->_" + c2.nextId);
42    }
43 }

```

Erläutern und begründen Sie jeweils, ob und warum es möglich bzw. unmöglich ist, dass das Programm das angegebene Verhalten zeigt.

Verhalten	Erläuterung, ob/warum Verhalten möglich/nicht möglich ist
Ausgabe c1: 42 -> 1337 c2: 1337 -> 42	
Ausgabe c1: 42 -> 0 c2: 1337 -> 0	
Nichtterminierung	

Aufgabe 2 (Schreibtischlauf)

(12 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten, während der Programmausführung keine Ausnahmen auftreten und etwaige Ausgaben nicht blockieren.

a) Gegeben sei folgendes paralleles Programm:

<pre>1 public class SL1 { 2 double answer = 0; 3 4 class T extends Thread{ 5 public void run(){ 6 answer = 42.13370815; 7 } 8 }</pre>	<pre>9 10 public void execute(){ 11 T t = new T(); 12 t.start(); 13 System.out.println(answer); 14 } 15 }</pre>
---	---

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort, wo dies verlangt ist.

i) Kann das Programm die Ausgabe `42.13370815` erzeugen?

, weil

ii) Kann das Programm die Ausgabe `0.0` erzeugen?

, weil

iii) Ist eine andere Ausgabe als die beiden oben genannten möglich?

, weil

iv) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

b) Gegeben sei folgendes paralleles Programm:

```

1  public class SL2 {
2      int a,b,c;
3      AtomicInteger d =
4          new AtomicInteger(0);
5      ReentrantLock lock =
6          new ReentrantLock();
7      CyclicBarrier barrier =
8          new CyclicBarrier(3);
9
10     public void execute()
11         throws Exception{
12         new T().start(); new T().start();
13
14         barrier.await();
15         System.out.println("a:_ " + a);
16
17         barrier.await();
18         System.out.println("b:_ " + b);
19
20         barrier.await();
21         System.out.println("c:_ " + c);
22
23         barrier.await();
24         System.out.println(
25             "d:_ " + d.get());
26         barrier.await();
27     }

```

```

28     class T extends Thread{
29     public void run(){
30         try {
31             a++;
32             barrier.await();
33             lock.lock();
34             b += a;
35             lock.unlock();
36             barrier.await();
37             synchronized (this){
38                 c++;
39             }
40             barrier.await();
41             d.set(d.get()+1);
42             barrier.await();
43         } catch (Exception e){}
44     }
45 }
46 }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort, wo dies verlangt ist.

i) Kann das Programm hinter „a:“ 3 ausgeben?

, weil

ii) Kann das Programm hinter „a:“ 1 ausgeben?

, weil

iii) Kann das Programm hinter „b:“ 0 ausgeben?

, weil

iv) Kann das Programm hinter „b:“ 1 ausgeben?

, weil

v) Kann das Programm hinter „c:“ 1 ausgeben?

, weil

vi) Kann das Programm hinter „d:“ 1 ausgeben?

, weil

vii) Kann es passieren, dass das Programm nicht terminiert?

, weil

Aufgabe 2 (Schreibtischlauf)

(11 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten, während der Programmausführung keine Ausnahmen auftreten und etwaige Ausgaben nicht blockieren.

a) Gegeben sei folgendes paralleles Programm:

```
1 public class SL1 {
2     int dat = 0;
3
4     class T extends Thread {
5         public void run() {
6             incrDat();
7         }
8     }
9
10    synchronized void incrDat() {
11        dat++;
12    }
13
14    synchronized int awaitAndGet() {
15        while(dat == 0) {}
16        return dat;
17    }
18
19    public void execute() throws Exception {
20        T t1 = new T(); t1.start();
21        T t2 = new T(); t2.start();
22        int datBuf = awaitAndGet();
23        t1.join(); t2.join();
24        System.out.println("dat:_" + datBuf);
25    }
}
```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort.

i) Ist es möglich, dass das Programm die Ausgabe `dat: 1` erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe `dat: 0` erzeugt?

, weil

iii) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

b) Gegeben sei folgendes paralleles Programm:

```

1 public class SL2 {
2     volatile int a = 0;
3     volatile int b = 0;
4     int c = 0;
5     volatile boolean t1Done = false;
6     volatile boolean t2Done = false;
7     int counter = 0;
8
9     class T1 extends Thread {
10    ... public void run() {
11        a = 1;
12        synchronized (SL2.class) {
13            counter++;
14        }
15        c++;
16        t1Done = true;
17    ... }
18    }
19
20    class T2 extends Thread {
21    ... public void run() {
22        try { Thread.sleep(1000);
23        } catch (Exception e) {}
24        b = a;
25        synchronized (SL2.class) {
26            counter++;
27        }
28        c++;
29        t2Done = true;
30    ... }
31    }
32
33    public void execute()
34    throws Exception {
35        new T1().start();
36        new T2().start();
37        while (true) {
38            synchronized (this) {
39                if(counter > 0) break;
40            }
41        }
42        while (!(t1Done && t2Done)) {}
43        System.out.println("b:_ " + b);
44        System.out.println("c:_ " + c);
45        System.out.println("counter:_ "
46            + counter);
47    }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort.

i) Kann das Programm die Ausgabe b: 0 erzeugen?

, weil

ii) Kann das Programm die Ausgabe c: 1 erzeugen?

, weil

iii) Kann das Programm die Ausgabe `c: 0` erzeugen?

, weil

iv) Kann das Programm die Ausgabe `counter: 1` erzeugen?

, weil

v) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

Aufgabe 4 (Schreibtischlauf)

(13 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) jeweils möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten, während der Programmausführung keine Ausnahmen auftreten und etwaige Ausgaben nicht blockieren.

Erinnerung: Die Methode `exchange` der Klasse `Exchanger` blockiert, ähnlich einer Barriere, bis sie von genau zwei Threads aufgerufen wurde. Sie liefert jedem aufrufenden Thread den Wert zurück, den der jeweils andere übergeben hat.

a) Gegeben sei folgendes paralleles Programm:

```

1 public class SL1 {
2     Exchanger<Integer> ex
3         = new Exchanger<>();
4     int a,b;
5
6     class Worker extends Thread {
7         synchronized void setValues(){
8             b = 1;
9             a = 1;
10        }
11
12        public void run() {
13            try {
14                setValues();
15                int sum = a + b +
16                    ex.exchange(0);
17                ex.exchange(sum);
18            } catch (Exception e) {}
19        }
20    }
21
22    void execute() throws Exception {
23        Thread w = new Worker();
24        w.start();
25        int la = 0;
26        while(la == 0) {
27            la = a;
28        }
29        int lb = b;
30        int buf = ex.exchange(40);
31        System.out.println(
32            "b:_" + b);
33        System.out.println(
34            "la:_" + la + "_lb:_" + lb);
35        System.out.println(
36            "sum:_" + ex.exchange(buf));
37    }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort.

i) Ist es möglich, dass das Programm die Ausgabe `b: 0` erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe `la: 0 lb: 1` erzeugt?

, weil

iii) Ist es möglich, dass das Programm die Ausgabe `1a: 1 1b: 0` erzeugt?

, weil

iv) Kann das Programm hinter `sum:` eine andere Zahl als 42 ausgeben? Falls ja, welche?

, weil

v) Kann es passieren, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

b) Gegeben sei folgendes paralleles Programm:

```

1 public class SL2 {
2     BlockingQueue<Integer> work =
3         new LinkedBlockingQueue<>();
4     BlockingQueue<Integer> done =
5         new LinkedBlockingQueue<>();
6     Worker w1, w2;
7
8     class Worker extends Thread {
9         public void run() {
10             while (!work.isEmpty()) {
11                 try {
12                     int todo = work.take();
13                     done.put(todo * 2);
14                 } catch (Exception e) {}
15             }
16         }
17     }
18
19     void execute() throws Exception {
20         w1 = new Worker(); w1.start();
21         w2 = new Worker(); w2.start();
22         for (int i = 1; i < 5; i++) {
23             work.put(i);
24         }
25         w1.join(); w2.join();
26         System.out.println("out:");
27         Integer out = done.poll();
28         while (out != null) {
29             System.out.print(out + " ");
30             out = done.poll();
31         }
32         System.out.println("done");
33     }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort.

i) Ist es möglich, dass das Programm die Ausgabe out : 2 6 8 4 done erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe out : 8 6 4 2 done erzeugt?

, weil

iii) Ist es möglich, dass das Programm die Ausgabe out : 2 4 done erzeugt?

, weil

iv) Ist es möglich, dass das Programm nicht terminiert?

, weil

Aufgabe 4 (Schreibtischlauf)

(8 Punkte)

Bestimmen Sie in dieser Aufgabe für das vorgegebene, syntaktisch korrekte parallele Programm, welches Verhalten (Ausgaben, Nichtterminierung) möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten, während der Programmausführung keine Ausnahmen auftreten und etwaige Ausgaben nicht blockieren.

```

1  public class SL1 {
2      volatile int a;
3      int b,c,e;
4      AtomicInteger d =
5          new AtomicInteger(0);
6      Object o = new Object();
7
8      void execute() throws Exception {
9          Worker w1 = new Worker();
10         Worker w2 = new Worker();
11         w1.start(); w2.start();
12         w1.join(); w2.join();
13         System.out.println("a:_ " + a +
14             "_b:_ " + b + "_c:_ " + c +
15             "_d:_ " + d + "_e:_ " + e);
16     }

```

```

17     class Worker extends Thread {
18
19         public void run() {
20             a++;
21             synchronized (this) {
22                 b++;
23             }
24             synchronized (Worker.class) {
25                 c++;
26             }
27             d.incrementAndGet();
28             synchronized (o) {
29                 o = new Object();
30                 e++;
31             }
32         }
33     }
34 }
35

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort ausführlich und verweisen Sie in Ihren Begründungen ggf. auf die Zeilennummern im Code, auf die Sie sich beziehen.

i) Ist es möglich, dass das Programm die Ausgabe a: 1 erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe a: 0 erzeugt?

, weil

iii) Ist es möglich, dass das Programm die Ausgabe $b: 1$ erzeugt?

, weil

iv) Ist es möglich, dass das Programm die Ausgabe $c: 1$ erzeugt?

, weil

v) Ist es möglich, dass das Programm die Ausgabe $d: 1$ erzeugt?

, weil

vi) Ist es möglich, dass das Programm die Ausgabe $e: 1$ erzeugt?

, weil

vii) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

Aufgabe 4 (Schreibtischlauf)

(13 Punkte)

Bestimmen Sie in dieser Aufgabe für die vorgegebenen, syntaktisch korrekten parallelen Programme, welches Verhalten (Ausgaben, Nichtterminierung) möglich ist. Die Ausführung beginnt in der Methode `execute`, die nicht nebenläufig aufgerufen wird. Nehmen Sie an, dass alle Instanzvariablen den initial zugewiesenen Wert enthalten, während der Programmausführung keine Ausnahmen auftreten und etwaige Ausgaben nicht blockieren.

a) Gegeben sei folgendes paralleles Programm:

<pre> 1 public class SL1 { 2 CyclicBarrier bar = 3 new CyclicBarrier(3); 4 int a = 0, c = 0, d = 0; 5 AtomicInteger b = 6 new AtomicInteger(0); 7 ReentrantLock lock = 8 new ReentrantLock(); 9 10 void execute() throws Exception { 11 Worker w1 = new Worker(); 12 Worker w2 = new Worker(); 13 w1.start(); w2.start(); 14 bar.await(); 15 System.out.println("a:_ 16 + a); 17 bar.await(); 18 System.out.println("b:_ 19 + b.get()); 20 21 bar.await(); 22 bar = new CyclicBarrier(3); 23 System.out.println("c:_ 24 + c); 25 bar.await(); 26 System.out.println("d:_"+ d); 27 } </pre>	<pre> 28 class Worker extends Thread { 29 public void run() { 30 try { 31 int tmp = a + 1; 32 bar.await(); 33 a = tmp; 34 b.set(b.get() + 1); 35 bar.await(); 36 synchronized (b) { 37 c++; 38 } 39 bar.await(); 40 lock.lock(); 41 d++; 42 lock.unlock(); 43 bar.await(); 44 } catch (Exception e) {} 45 } 46 } 47 } </pre>
--	--

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort ausführlich und verweisen Sie in Ihren Begründungen ggf. auf die Zeilennummern im Code, auf die Sie sich beziehen.

i) Ist es möglich, dass das Programm die Ausgabe `a: 0` erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe **a: 2** erzeugt?

, weil

iii) Ist es möglich, dass das Programm die Ausgabe **b: 1** erzeugt?

, weil

iv) Ist es möglich, dass das Programm die Ausgabe **c: 1** erzeugt?

, weil

v) Ist es möglich, dass das Programm die Ausgabe **d: 1** erzeugt?

, weil

vi) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil

b) Gegeben sei folgendes paralleles Programm:

```

1  public class SL2 {
2      long value = 0;
3
4      synchronized void setVal(long in) {
5          value = in;
6      }
7
8      void execute() {
9          Doubler d = new Doubler();
10         d.start();
11         synchronized (this) {
12             d.process(20);
13         }
14         synchronized (this) {
15             d.process(21);
16         }
17         System.out.println("value:_"
18             + value);
19     }
20
21     class Doubler extends Thread {
22         volatile long i = 0;
23
24         synchronized void process
25             (long in){
26             i = in;
27         }
28
29         public void run(){
30             while(value != 42) {
31                 synchronized (this) {
32                     setVal(i * 2);
33                 }
34             }
35         }
36     }
37
38 }

```

Beantworten Sie die folgenden Fragen zu diesem Programm und dessen möglichen Ausgaben. Begründen Sie Ihre Antwort ausführlich und verweisen Sie in Ihren Begründungen ggf. auf die Zeilennummern im Code, auf die Sie sich beziehen.

i) Ist es möglich, dass das Programm die Ausgabe `value: 0` erzeugt?

, weil

ii) Ist es möglich, dass das Programm die Ausgabe `value: 40` erzeugt?

, weil

iii) Ist es möglich, dass das Programm nicht terminiert und keine Ausgabe erzeugt?

, weil